Chapter 10: Constraints

Algorithms for Optimization, 2nd Ed.
Kochenderfer & Wheeler (2026)

Lecture Slides

Chapter Outline

Why Constraints Arise I

All of the optimization methods studied in previous chapters assume that the decision variable x can take *any* value in $\mathbb{R}^n$ (the set of all real n-dimensional vectors). In practice, however, the physical or economic reality of a problem almost always restricts x to a smaller set of **feasible** values.

Real-world examples where constraints are unavoidable

- **Structural mechanics.** The stress σ (sigma) inside an aircraft wing cannot exceed a material limit $\sigma_{\max}$ (sigma sub max), otherwise the wing fails. This gives the inequality constraint $\sigma \leq \sigma_{\max}$.
- **Portfolio allocation.** If w_i is the fraction of money invested in asset i, then the fractions must sum to exactly one (a budget equality constraint): $\sum_{i=1}^{n} w_i = 1$. Here $\sum$ (Sigma) denotes the sum over all assets $i = 1, \ldots, n$.
- **Robot kinematics.** Each joint has physical hard stops, so the joint angle θ (theta) is bounded: $\theta_{\min} \leq \theta \leq \theta_{\max}$.

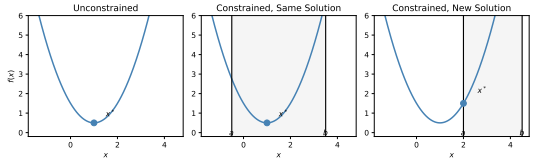
The set of all x values that satisfy the constraints is called the **feasible set** $\mathcal{X}$. When $\mathcal{X} \subsetneq \mathbb{R}^n$ (strictly smaller than the full space), the problem is **constrained**.

Why Constraints Arise II

Adding a constraint does one of two things:

1. **It changes the optimum.** The unconstrained minimum falls outside the feasible set, so we are forced to accept a worse (but feasible) solution at the boundary of $\mathcal{X}$.
2. **It leaves the optimum unchanged.** The unconstrained minimum already satisfies all constraints, so imposing them has no effect.

The right-hand figure illustrates both cases. The contours (level curves) of the objective f are shown, together with a constraint boundary (a line). In the middle panel the unconstrained minimum lies inside the feasible region, so the constraint is *inactive*. In the right panel the minimum lies outside, so the constraint is *active* and the solution moves to the boundary.



A constraint can shift the optimum to the boundary (right panel) or leave it unchanged when it is already feasible (middle panel).

Why Constraints Arise III

Key Insight

Whether a constraint "matters" depends entirely on where the unconstrained minimum sits relative to the feasible set. A core goal of this chapter is to develop methods that efficiently find the constrained optimum regardless of which case applies.

The General Constrained Optimization Problem I

The most common way to write a constrained optimization problem unifies all constraint types into a single *standard form*.

Standard Form (Eq. 10.2)

$$\underset{\mathbf{x}}{\text{minimize}} \; f(\mathbf{x}) \qquad \text{subject to} \quad \begin{aligned} h_i(\mathbf{x}) &= 0, \quad i = 1, \ldots, \ell \\ g_j(\mathbf{x}) &\leq 0, \quad j = 1, \ldots, m \end{aligned}$$

Notation

- $\mathbf{x} \in \mathbb{R}^n$ (x bold) is the **design vector** — the n variables we are optimizing over.
- $f : \mathbb{R}^n \to \mathbb{R}$ is the **objective function** — the scalar quantity we want to minimize.
- $h_i : \mathbb{R}^n \to \mathbb{R}$ (h sub i) are the ℓ (ell) **equality constraints**. Requiring $h_i(\mathbf{x}) = 0$ pins x to a surface of dimension $n - \ell$.
- $g_j : \mathbb{R}^n \to \mathbb{R}$ (g sub j) are the m **inequality constraints**. The convention $g_j(\mathbf{x}) \leq 0$ means the constraint is satisfied when the function value is zero or negative.

How to Read This Formula

Reading the standard form aloud: "minimize $f(x)$ over all choices of x, subject to the requirement that each equality function h_i equals zero, and each inequality function g_j is at most zero." The word *subject to* (often abbreviated *s.t.*) means "while also satisfying".

Converting other forms. At first glance it may seem that only $\leq$ inequalities are handled. In fact, any constraint can be rewritten in this form:

- A greater-than constraint $g(x) \geq 0$ is equivalent to $-g(x) \leq 0$, so simply negate the function.
- A strict equality $h(x) = 0$ is already in the form above.
- A set-membership requirement $x \in \mathcal{X}$ (for some geometric set $\mathcal{X}$) can often be converted to a collection of inequality functions.

Worked Example — writing a problem in standard form

Suppose we want to minimize $x_1^2 + x_2^2$ (the squared distance from the origin) subject to: the point lying on the line $x_1 + x_2 = 1$, and both coordinates being non-negative.

In standard form:

$$\begin{aligned} &\underset{x_1, x_2}{\text{minimize}}\ x_1^2 + x_2^2 \\ &\quad \text{s.t. } h(x_1, x_2) = x_1 + x_2 - 1 = 0 \quad \text{(equality)} \\ &\qquad g_1(x_1, x_2) = -x_1 \leq 0 \quad \text{(i.e. } x_1 \geq 0\text{)} \\ &\qquad g_2(x_1, x_2) = -x_2 \leq 0 \quad \text{(i.e. } x_2 \geq 0\text{)} \end{aligned}$$

Note that $x_i \geq 0$ becomes $-x_i \leq 0$ by negation.

A Taxonomy of Constraint Types I

While the standard form uses only equalities and $\leq$ inequalities, it is helpful to recognise the different geometric shapes these constraints produce, because some algorithms exploit that geometry directly.

1. Equality Constraints — $h(\mathrm{x}) = 0$

An equality constraint restricts x to a **surface** (technically a manifold) embedded in $\mathbb{R}^n$. With ℓ independent equality constraints, the feasible surface has dimension $n - \ell$. For instance, with $n = 3$ variables and one equality constraint ($\ell = 1$), the feasible set is a two-dimensional surface inside three-dimensional space.

2. Inequality Constraints — $g(\mathrm{x}) \leq 0$

An inequality constraint restricts x to one **side** of a surface: the set $\{ \mathrm{x} : g(\mathrm{x}) \leq 0 \}$ is a closed half-space (if g is linear) or a curved feasible region (if g is nonlinear). The boundary $g(\mathrm{x}) = 0$ is called the **constraint boundary**.

A Taxonomy of Constraint Types II

3. Bound and Box Constraints — $a \leq x_i \leq b$

A bound constraint on the i-th component can be written as two inequality constraints:

$$g_1(x_i) = x_i - b \leq 0 \quad \text{and} \quad g_2(x_i) = a - x_i \leq 0.$$

When bounds are placed on *every* component simultaneously, a $\leq$ x $\leq$ b, the feasible set is a **box** (a hyperrectangle in $\mathbb{R}^n$).

A Taxonomy of Constraint Types III

The following table summarises all constraint types and their unified representations in standard form.

Type	**Form**	**Geometric meaning**
Equality	$h(\mathrm{x}) = 0$	Surface / manifold
Inequality	$g(\mathrm{x}) \leq 0$	Half-space or curved region
Bound	$a \leq x_i \leq b$	Interval on one axis
Box	$\mathrm{x} \in [\mathrm{a}, \mathrm{b}]$	Hyperrectangle
Set-membership	$\mathrm{x} \in \mathcal{X}$	Any closed set

Key Insight

Because any constraint type can be converted to equality or inequality form, we only need to develop algorithms that handle these two types — and those algorithms will apply universally to all constrained problems.

Removing Bound Constraints via Variable Transformation I

One of the most elegant strategies for handling a bound constraint $a \leq x \leq b$ is to **transform the variable** so that the bound is automatically satisfied. We introduce an *unconstrained* helper variable $\hat{x}$ (x-hat) that can take any real value, and define x as a function $t_{a,b}(\hat{x})$ that always maps into (a, b). Because x can never violate the bound by construction, the constraint disappears from the problem statement entirely, and any unconstrained solver can be applied to the transformed problem.

The Sinusoidal Bound Transform

$$x = t_{a,b}(\hat{x}) = a + (b - a)\,\sin^2(\hat{x})$$

Removing Bound Constraints via Variable Transformation II

Notation and How to Read This

- a and b are the lower and upper bounds, respectively (e.g., $a = 2$, $b = 6$).
- $\hat{x}$ (x-hat) is the **unconstrained** helper variable; it can be any real number.
- $\sin^2(\hat{x})$ (sine squared of x-hat) always lies in $[0, 1]$ regardless of the value of $\hat{x}$, because sine is always between -1 and 1.
- Therefore $t_{a,b}(\hat{x})$ always lies in $[a, a + (b - a) \cdot 1] = [a, b]$, satisfying the bound automatically.

The formula maps an infinite real line onto the bounded interval $[a, b]$, wrapping back and forth like a sinusoid. Every value inside (a, b) is achieved by infinitely many values of $\hat{x}$.

Removing Bound Constraints via Variable Transformation III

Procedure. To minimize $f(x)$ subject to $a \le x \le b$:

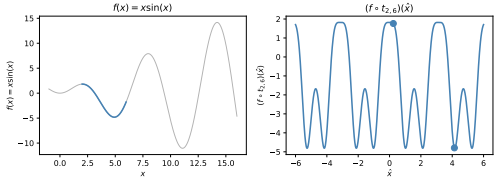
1. Define the helper variable $\hat{x} \in \mathbb{R}$ (unconstrained).
2. Substitute $x = t_{a,b}(\hat{x})$ into f to get a new function $\tilde{f}(\hat{x}) = f\big(t_{a,b}(\hat{x})\big)$.
3. Minimize $\tilde{f}$ over $\hat{x}$ using any unconstrained algorithm (gradient descent, L-BFGS, etc.).
4. Convert the solution $\hat{x}^*$ back to $x^* = t_{a,b}(\hat{x}^*)$.

Worked Example 10.1 — min $x\sin(x)$, $x \in [2, 6]$

The function $f(x) = x\sin(x)$ has several local minima on $[2, 6]$. After transformation, $\tilde{f}(\hat{x}) = t_{2,6}(\hat{x}) \cdot \sin\big(t_{2,6}(\hat{x})\big)$ is an unconstrained function on $\mathbb{R}$.

Running L-BFGS from multiple starting points finds two minima in $\hat{x}$-space at approximately $\hat{x} \approx 0.242$ and $\hat{x} \approx 4.139$, both mapping back to $x^* \approx 4.914$ in the original space.

The optimal value is $f^* \approx -4.814$.



Top: original $f(x)$ on $[2, 6]$. Bottom: transformed $\tilde{f}(\hat{x})$ on $\mathbb{R}$. Two local minima in $\hat{x}$ both correspond to the same x^*.

Removing Bound Constraints via Variable Transformation IV

Key Insight

The transformation trades a constrained problem for an unconstrained one, but it introduces *multiple pre-images* (several $\hat{x}$ values correspond to the same x), which can create spurious local minima. Running from multiple starting points mitigates this.

Python: Bound-Constrained Minimization via Transform

```python
import numpy as np
from scipy.optimize import minimize

a, b = 2.0, 6.0

def t(xhat):
    """Sinusoidal transform: maps any real xhat into (a, b)."""
    return a + (b - a) * np.sin(xhat)**2

def f_transformed(xhat):
    x = t(xhat)
    return x * np.sin(x)

# Solve unconstrained problem from many starting points to avoid local minima
results = []
for x0 in np.linspace(-6, 6, 20):
    res = minimize(f_transformed, x0, method='L-BFGS-B')
    results.append((res.fun, t(res.x[0])))

# Best solution in original space
best = min(results, key=lambda r: r[0])
print(f"f* = {best[0]:.4f},  x* = {best[1]:.4f}")
# Output: f* = -4.8141,  x* = 4.9136
```

Listing 1: Example 10.1 in Python — bound-constrained minimization using sinusoidal transform

Removing Affine Equality Constraints via LQ Decomposition I

An **affine equality constraint** has the form $Ax = b$, where $A \in \mathbb{R}^{m \times n}$ is a matrix of coefficients and $b \in \mathbb{R}^m$ is a constant vector. For example, "the sum of all portfolio weights equals one" is an affine constraint.

The idea is to perform a change of variables that **automatically satisfies** the equality, so the constraint can be dropped entirely. We do this using the **LQ decomposition** of A.

LQ Decomposition of A (Eq. 10.6)

$$A = \begin{bmatrix} L & 0 \end{bmatrix} Q$$

Removing Affine Equality Constraints via LQ Decomposition II

Notation

- $A \in \mathbb{R}^{m \times n}$ (A bold) is the constraint coefficient matrix, with m rows (one per equality constraint) and n columns (one per variable). We assume $m < n$ (fewer constraints than variables) and that A has full row rank m.
- $L \in \mathbb{R}^{m \times m}$ (L bold) is a **lower-triangular** matrix — meaning all entries above the main diagonal are zero. It captures how the constraints couple the variables.
- 0 is an $m \times (n - m)$ matrix of zeros, padding L to width n.
- $Q \in \mathbb{R}^{n \times n}$ (Q bold) is an **orthogonal** matrix — meaning its columns are perpendicular unit vectors, so $Q^\top Q = I$. It defines a rotation of the coordinate system.

Removing Affine Equality Constraints via LQ Decomposition III

How the substitution works. Define a rotated variable vector $y = Qx$ (y bold equals Q times x bold), so $x = Q^\top y$. Split y into two parts:

$$y = \begin{bmatrix} y_{1:m} \\ y_{(m+1):n} \end{bmatrix}$$

where $y_{1:m}$ denotes the first m (em) components and $y_{(m+1):n}$ denotes the remaining $n - m$ components. Substituting into the constraint $Ax = b$:

$$\begin{bmatrix} L & 0 \end{bmatrix} y = b \implies L\, y_{1:m} = b \implies y^*_{1:m} = L^{-1} b.$$

Since L is lower-triangular, $L^{-1}b$ is computed cheaply by forward substitution. The first m components of y are thus **fixed** by the constraint, and we only need to optimize over the **free** components $y_{(m+1):n}$.

Key Insight

The LQ decomposition splits the variables into "constrained" ones (fully determined by the equality) and "free" ones (which we optimize unconstrained). The problem dimension drops from n to $n - m$, which can be a major computational saving when m is large.

Removing Affine Equality Constraints via LQ Decomposition IV

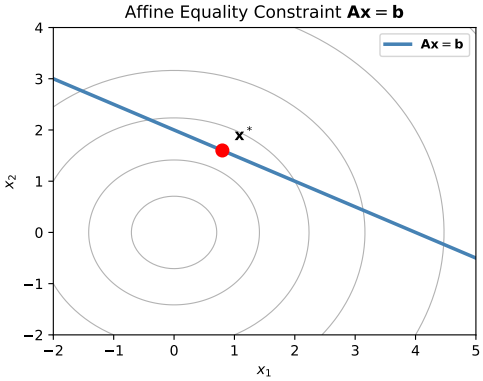
Example 10.2 — scalar equality constraint

Suppose the constraint is a single linear equation:
$c_1 x_1 + c_2 x_2 + \cdots + c_n x_n = 0$.

We can solve for the last variable:

$$x_n = \frac{1}{c_n}\left(-c_1 x_1 - c_2 x_2 - \cdots - c_{n-1} x_{n-1}\right)$$

and substitute this into f, eliminating x_n entirely. The resulting problem has $n-1$ free variables and no constraints.



The constraint $x_1 + 2x_2 = 4$ (dashed line) forces the optimum onto the line. The constrained minimum is $x^* = (4/5,\ 8/5)$.

Python: Affine Equality Constraint via Substitution

```python
import numpy as np
from scipy.optimize import minimize

# Constraint: x1 + 2*x2 + 3*x3 = 6  =>  x1 = 6 - 2*x2 - 3*x3
# We eliminate x1 and optimize over [x2, x3] only.
def f_reduced(y):
    """y = [x2, x3]; x1 is determined by the constraint."""
    x2, x3 = y
    x1 = 6.0 - 2*x2 - 3*x3
    # Original objective: x1^3 + x2^2 + x3  (substituted)
    return x1**3 + x2**2 + x3

x0 = [1.0, 1.0]
res = minimize(f_reduced, x0, method='L-BFGS-B')
x2_opt, x3_opt = res.x
x1_opt = 6.0 - 2*x2_opt - 3*x3_opt

print(f"x* = ({x1_opt:.4f}, {x2_opt:.4f}, {x3_opt:.4f})")
print(f"Constraint satisfied: {np.isclose(x1_opt + 2*x2_opt + 3*x3_opt, 6)}")
```

Listing 2: Minimize $x_1^3 + x_2^2 + x_3$ subject to $x_1 + 2x_2 + 3x_3 = 6$ by substitution

Projected Gradient Descent — Staying Feasible at Every Step I

When the feasible set $\mathcal{X}$ is a convex set (such as a box or a half-space), a natural extension of gradient descent is the **projected gradient descent** algorithm. The key idea is:

1. Take a normal gradient step (which might leave the feasible set).
2. **Project** the new point back onto $\mathcal{X}$ — i.e., snap it to the nearest feasible point.

Because both the gradient step and the projection are computable operations, the algorithm is simple to implement and guarantees that every iterate is feasible.

The Projection Operator

$$\Pi_{\mathcal{X}}(\tilde{x}) = \arg\min_{x \in \mathcal{X}} \|x - \tilde{x}\|_2$$

Projected Gradient Descent — Staying Feasible at Every Step II

Notation

- $\Pi_{\mathcal{X}}$ (Pi sub X, where Π is capital pi) denotes the **projection** operator onto $\mathcal{X}$.
- $\tilde{x}$ (x-tilde) is the **tentative** point — the result of the gradient step, which may be infeasible.
- $\|x - \tilde{x}\|_2$ (norm of x minus x-tilde) is the Euclidean distance between two points.
- The projection finds the point in $\mathcal{X}$ that is *closest* (in Euclidean distance) to $\tilde{x}$.

Projected Gradient Descent — Staying Feasible at Every Step III

How to Read the Projection Formula

Think of $\Pi_{\mathcal{X}}(\tilde{\mathbf{x}})$ as the answer to: "If I am standing at $\tilde{\mathbf{x}}$ (which might be outside the allowed region), what is the closest feasible point?" For a box constraint $[a_i, b_i]$ on each component, the answer is simply to **clip** each coordinate: $x_i = \max(a_i, \min(b_i, \tilde{x}_i))$. For more complex sets, finding the projection requires solving a small quadratic program (QP).

Algorithm 10.1 — Projected Gradient Descent

1. Choose a feasible starting point $\mathbf{x}^{(1)} \in \mathcal{X}$. (Here $\mathbf{x}^{(1)}$ means x superscript 1, the value at iteration 1.)
2. At iteration k (kay), compute the **gradient step**:

$$\tilde{\mathbf{x}} \leftarrow \mathbf{x}^{(k)} - \alpha \nabla f\big(\mathbf{x}^{(k)}\big)$$

   where α (alpha) is the step size (a small positive number) and ∇f (∇ is nabla, meaning gradient) is the vector of partial derivatives of f.
3. **Project** back into $\mathcal{X}$:

$$\mathbf{x}^{(k+1)} \leftarrow \Pi_{\mathcal{X}}(\tilde{\mathbf{x}})$$

4. Repeat steps 2–3 until $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|$ is smaller than a convergence tolerance.

Projected Gradient Descent — Staying Feasible at Every Step V

Special cases for the projection step

Box constraint [a, b]: the projection is componentwise clipping,

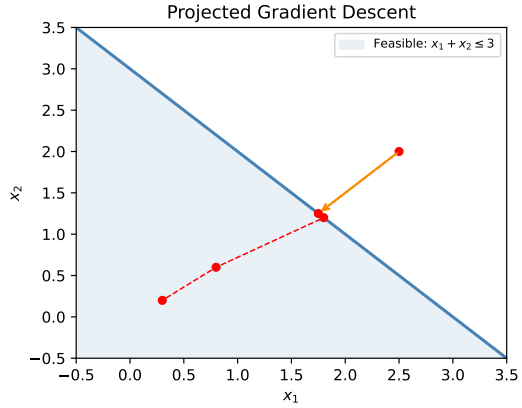
$$\left[\Pi_{[\mathrm{a},\mathrm{b}]}(\tilde{\mathrm{x}})\right]_i = \max(a_i,\, \min(b_i,\, \tilde{x}_i)).$$

This is $O(n)$ (order n) — linear in the number of variables.

Half-space $\{\mathrm{x} : \mathrm{c}^\top \mathrm{x} \le d\}$ (c bold transpose x $\le$ d): if $\tilde{\mathrm{x}}$ violates the constraint, shift it back along the constraint normal c:

$$\Pi(\tilde{\mathrm{x}}) = \tilde{\mathrm{x}} - \frac{(\mathrm{c}^\top \tilde{\mathrm{x}} - d)}{\|\mathrm{c}\|^2}\,\mathrm{c}.$$

General convex set: solve a small QP (quadratic program) at each iteration.



Orange arrows: unconstrained gradient steps that move outside the feasible region. Red path: the projected iterates, which walk along the constraint boundary toward the constrained minimum.

Projected Gradient Descent — Staying Feasible at Every Step VI

Key Insight

Projected gradient descent guarantees that *every* iterate is feasible, which is useful when the objective or constraints are undefined outside $\mathcal{X}$. The price is the projection step, which itself requires solving an optimization sub-problem for non-trivial sets.

Python: Projected Gradient Descent

```python
import numpy as np

def project_halfspace(x, a, b_val):
    """Project x onto the half-space {x : a^T x <= b_val}.
    If x already satisfies the constraint, it is returned unchanged.
    Otherwise, x is shifted back onto the boundary along direction a."""
    violation = a @ x - b_val   # positive means infeasible
    if violation > 0:
        x = x - violation * a / (a @ a)
    return x

def f(x):       return (x[0]-3)**2 + (x[1]-3)**2
def grad_f(x): return np.array([2*(x[0]-3), 2*(x[1]-3)])

a = np.array([1.0, 1.0])    # constraint normal vector
b_val = 3.0                  # constraint right-hand side
x = np.array([2.5, 2.0])    # starting point (infeasible)
alpha = 0.1                  # step size (alpha)

for _ in range(50):
    x_new = x - alpha * grad_f(x)        # gradient step
    x = project_halfspace(x_new, a, b_val)  # project back

print(f"x* = {x},  constraint satisfied: {a @ x <= b_val + 1e-8}")
# x* approximately [1.5, 1.5],  constraint satisfied: True
```

Listing 3: Projected gradient descent onto the half-space $x_1 + x_2 \leq 3$

Lagrange Multipliers — the Geometry of Equality Constraints I

When a constraint $h(\mathrm{x}) = 0$ is present, the optimal point x^* must satisfy a special geometric condition: the **gradient of the objective** must be **parallel** to the **gradient of the constraint**. Intuitively, if $\nabla f(\mathrm{x}^*)$ had a component *along* the constraint surface, we could move a tiny distance along the surface (remaining feasible) and decrease f — contradicting optimality.

First-Order Optimality Condition for One Equality Constraint (Eq. 10.15)

$$\nabla f(\mathrm{x}^*) = \lambda \nabla h(\mathrm{x}^*)$$

Lagrange Multipliers — the Geometry of Equality Constraints II

Notation

- $\nabla f(x^*)$ (∇ is nabla, meaning gradient) is the vector of partial derivatives of f evaluated at the optimal point x^*. It points in the direction of steepest increase of f.
- $\nabla h(x^*)$ is the gradient of the constraint function h at x^*. It is perpendicular to the constraint surface $h(x) = 0$.
- λ (lambda) is the **Lagrange multiplier** — a scalar that records how much the constraint gradient must be "stretched" to match the objective gradient. Its sign and magnitude convey economic information (the "shadow price" of relaxing the constraint by one unit).

The Lagrangian function. Rather than working with the geometric condition directly, it is more convenient to define a single scalar function whose unconstrained critical points encode *both* the optimality condition and the constraint.

The Lagrangian (Eq. 10.16)

$$\mathcal{L}(\mathrm{x}, \lambda) = f(\mathrm{x}) - \lambda\, h(\mathrm{x})$$

Notation and How to Read This

- $\mathcal{L}$ (script L, called the "Lagrangian") is a function of *both* x and λ (lambda).
- Setting the partial derivative with respect to x to zero, $\nabla_x \mathcal{L} = 0$, gives the condition $\nabla f(x) = \lambda \nabla h(x)$ — exactly the optimality condition above.
- Setting the partial derivative with respect to λ (lambda) to zero, $\partial \mathcal{L} / \partial \lambda = 0$, gives $h(x) = 0$ — i.e., it *enforces the constraint*.

In other words, solving the unconstrained problem $\nabla \mathcal{L}(x, \lambda) = 0$ simultaneously finds the optimal x^* and the Lagrange multiplier λ^*.

Example 10.4 — Numerical Lagrange Multiplier Solution

Problem:

$$\min_{x} \; -\exp\Big(-\big(x_1x_2-\tfrac{3}{2}\big)^2-\big(x_2-\tfrac{3}{2}\big)^2\Big) \quad \text{s.t.} \quad x_1-x_2^2=0.$$

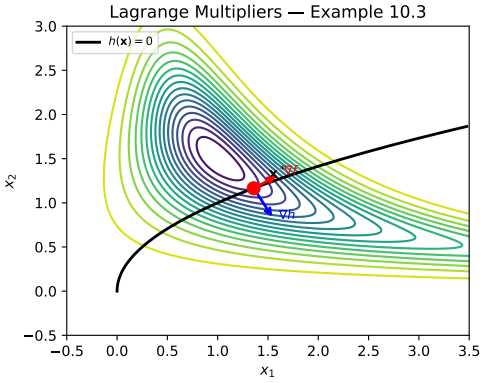
The Lagrangian is:

$$\mathcal{L} = -\exp(\cdots) - \lambda\,(x_1 - x_2^2).$$

Setting all partial derivatives of $\mathcal{L}$ to zero and solving numerically yields:

$$x^* \approx [1.358,\ 1.165], \qquad \lambda^* \approx 0.170.$$

The positive λ^* (lambda star) means that relaxing the constraint (allowing $x_1 - x_2^2$ to be slightly positive) would improve the objective by approximately 0.170 per unit of



At $x^* \approx [1.358, 1.165]$, a level curve (contour) of f is tangent to the constraint curve $x_1 = x_2^2$. Tangency means the gradients ∇f and ∇h are parallel.

Multiple Equality Constraints I

When there are ℓ (ell) equality constraints $h_1(\mathrm{x}) = 0, \ldots, h_\ell(\mathrm{x}) = 0$, we introduce one Lagrange multiplier λ_i (lambda sub i) for each constraint.

Generalized Lagrangian for Multiple Equalities

$$\mathcal{L}(\mathrm{x}, \boldsymbol{\lambda}) = f(\mathrm{x}) - \sum_{i=1}^{\ell} \lambda_i \, h_i(\mathrm{x})$$

where $\boldsymbol{\lambda} = (\lambda_1, \ldots, \lambda_\ell)$ (lambda bold) is the vector of all Lagrange multipliers.

Multiple Equality Constraints II

Notation

- $\sum_{i=1}^{\ell}$ (Sigma from $i = 1$ to ℓ) denotes the sum of ℓ terms. Each term $\lambda_i h_i(\mathrm{x})$ is the product of multiplier i and constraint function i.
- Setting $\nabla_{\mathrm{x}} \mathcal{L} = 0$ yields the stationarity condition:

$$\nabla f(\mathrm{x}) = \sum_{i=1}^{\ell} \lambda_i \nabla h_i(\mathrm{x}),$$

  which says that ∇f must lie in the span (linear combination) of the constraint gradients.
- Setting $\partial \mathcal{L} / \partial \lambda_i = 0$ for each i enforces $h_i(\mathrm{x}) = 0$ for each constraint separately.

Multiple Equality Constraints III

How to Read the Stationarity Condition

The condition $\nabla f = \sum_i \lambda_i \nabla h_i$ generalises the single-constraint case. It says: at an optimal point, the gradient of the objective can be "explained" as a weighted combination of the constraint gradients. If this fails — i.e., if ∇f has a component that cannot be expressed as such a combination — then we could improve f while remaining feasible, contradicting optimality.

Python: Method of Lagrange Multipliers (Numerical)

```python
import numpy as np
from scipy.optimize import fsolve

def f(x):
    """Objective: -exp(-(x1*x2 - 3/2)^2 - (x2 - 3/2)^2)"""
    return -np.exp(-((x[0]*x[1] - 1.5)**2 + (x[1] - 1.5)**2))

def grad_L(vars):
    """Gradient of the Lagrangian L(x1, x2, lambda) = f(x) - lambda*(x1 - x2^2).
    We set all three partial derivatives to zero and solve the system."""
    x1, x2, lam = vars
    fval = f([x1, x2])
    # dL/dx1: derivative of f w.r.t. x1 minus lambda*(d/dx1)(x1 - x2^2)
    dLdx1 = 2*x2*fval*(x1*x2 - 1.5) - lam
    # dL/dx2: derivative of f w.r.t. x2 minus lambda*(d/dx2)(x1 - x2^2)
    dLdx2 = 2*lam*x2*fval + fval*(-2*x1*(x1*x2-1.5) - 2*(x2-1.5))
    # dL/dlambda: enforces the constraint h(x) = x1 - x2^2 = 0
    dLdlam = -(x2**2 - x1)
    return [dLdx1, dLdx2, dLdlam]

x0 = [1.0, 1.0, 0.1]           # initial guess for (x1, x2, lambda)
sol = fsolve(grad_L, x0, full_output=True)
x1s, x2s, lams = sol[0]
print(f"x* = [{x1s:.4f}, {x2s:.4f}],  lambda* = {lams:.4f}")
# x* = [1.3580, 1.1650],  lambda* = 0.1700
```

Listing 4: Solve the KKT system for Example 10.4 using fsolve — find x* and lambda*

KKT Conditions — the Full First-Order Theory I

The Lagrange multiplier theory extends elegantly to inequality constraints $g_j(\mathrm{x}) \leq 0$ by introducing one multiplier μ_j (mu sub j) per inequality. The resulting **Karush-Kuhn-Tucker (KKT) conditions** are the first-order necessary conditions for a constrained minimum of any smooth problem.

Generalized Lagrangian with Equality and Inequality Constraints

$$\mathcal{L}(\mathrm{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\mathrm{x}) - \sum_{i=1}^{\ell} \lambda_i h_i(\mathrm{x}) - \sum_{j=1}^{m} \mu_j g_j(\mathrm{x})$$

Notation

- $\boldsymbol{\lambda} = (\lambda_1, \ldots, \lambda_\ell)$ (lambda bold) are the multipliers for the ℓ equality constraints; they are *unrestricted* in sign.
- $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_m)$ (mu bold) are the multipliers for the m inequality constraints; they must be **non-negative** ($\mu_j \geq 0$) — this is the dual feasibility condition explained below.
- $\sum_{j=1}^{m}$ (Sigma from $j = 1$ to m) sums over all m inequality constraints.

The Four KKT Necessary Conditions

At any local minimum x^* (under a constraint qualification):

1. **Stationarity:** $\nabla_x \mathcal{L}(x^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) = 0$. The gradient of the Lagrangian with respect to x (x bold) must be zero, just as in unconstrained optimization.
2. **Primal feasibility:** $h_i(x^*) = 0$ for all i, and $g_j(x^*) \leq 0$ for all j. The optimal point must satisfy all constraints.
3. **Dual feasibility:** $\mu_j^* \geq 0$ for all j. The multipliers for inequality constraints must be non-negative. This ensures that the constraint gradient pushes the objective in the correct direction (into the feasible set).
4. **Complementary slackness:** $\mu_j^* \, g_j(x^*) = 0$ for all j. For each inequality, *either* the multiplier is zero *or* the constraint is tight (equals zero), but both cannot be non-zero simultaneously.

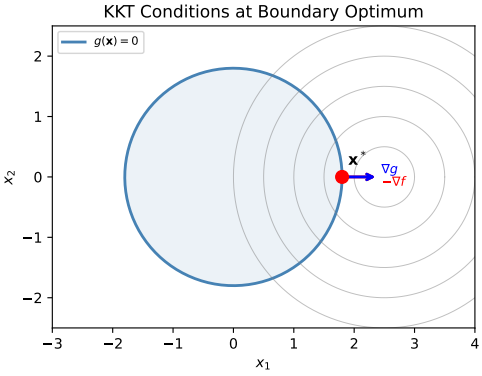
KKT Conditions — the Full First-Order Theory III

How to Read Complementary Slackness

The condition $\mu_j g_j(x^*) = 0$ encodes the two possible situations at an optimal point:

- **Inactive constraint** ($g_j(x^*) < 0$): the point is strictly inside the feasible set. Then $\mu_j^* = 0$; the constraint has no influence on the solution.
- **Active constraint** ($g_j(x^*) = 0$): the point sits exactly on the boundary. Then $\mu_j^* > 0$; the constraint is "binding" and pulls the solution to the boundary.

The product of these two quantities must always be zero: if $g_j < 0$, we need $\mu_j = 0$; if $\mu_j > 0$, we need $g_j = 0$.



At a boundary optimum: $-\nabla f$ and ∇g are aligned ($\mu^* > 0$). At an interior optimum: the constraint gradient is not needed ($\mu^* = 0$).

Active vs. Inactive Constraints — Worked Intuition I

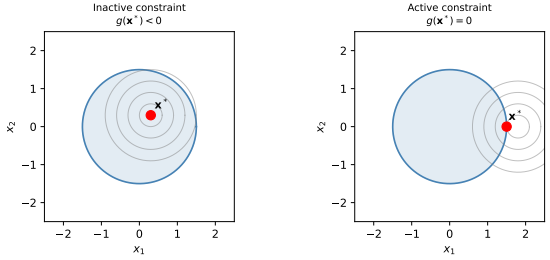
Understanding whether a constraint is active or inactive at the solution is crucial for selecting efficient algorithms. The distinction maps directly onto the complementary slackness KKT condition.

Inactive Constraint — $g(x^*) < 0$

When the unconstrained minimum already satisfies $g(x^*) < 0$ (strictly inside the feasible region), the constraint has no effect on the solution. The KKT condition forces $\mu_j^* = 0$, meaning the corresponding multiplier is zero. Locally, the problem behaves exactly like an unconstrained one.

Active Constraint — $g(x^*) = 0$

When the unconstrained minimum is infeasible (i.e., $g(x) < 0$ is violated at the unconstrained optimum), the constrained minimum is forced to the boundary $g(x^*) = 0$. The KKT condition gives $\mu_j^* > 0$, meaning the constraint is "binding": the gradient of



Left panel: the unconstrained minimum lies inside the feasible region, so the constraint is inactive ($\mu = 0$). Right panel: the minimum lies outside the feasible region, so it moves to the boundary (active constraint, $\mu > 0$).

Active vs. Inactive Constraints — Worked Intuition II

Key Insight

At an optimal point, each inequality constraint is either *on* or *off*: either it is active and pushing the solution to the boundary, or it is inactive and irrelevant. This binary behaviour is exploited by **active-set methods**, which maintain a working set of active constraints and update it as the algorithm progresses.

Slack Variables — Converting Inequalities to Equalities I

A **slack variable** is a device that converts an inequality constraint into an equality constraint by absorbing the "slack" — the gap between the left-hand side and zero. This is useful because some methods (e.g., Lagrange multipliers, the method of multipliers) are designed for equality constraints only.

The Slack Variable Trick

Given an inequality $g(\mathrm{x}) \leq 0$, introduce a new variable $s \geq 0$ (the slack) so that:

$$g(\mathrm{x}) + s = 0, \qquad s \geq 0.$$

The slack s measures how far the constraint is from being tight: if $s = 0$, the constraint is active; if $s > 0$, the constraint is inactive with slack s.

The remaining problem is: how do we enforce $s \geq 0$ without another inequality constraint? We use a further substitution $s = t^2$, where $t \in \mathbb{R}$ is unconstrained. Then $s = t^2 \geq 0$ automatically.

Slack Variables — Converting Inequalities to Equalities II

Full Transformation (no constraints on new variables)

$$g(\mathrm{x}) + t^2 = 0, \qquad t \in \mathbb{R} \text{ (unconstrained)}.$$

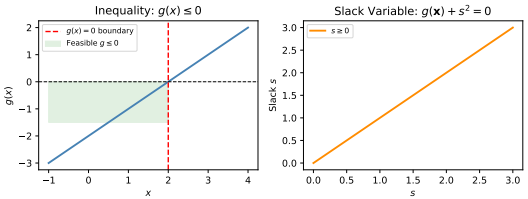
The original inequality $g(\mathrm{x}) \leq 0$ has been converted to an equality constraint involving both x and the new variable t.

Slack Variables — Converting Inequalities to Equalities III

Procedure for m inequality constraints. Introduce m unconstrained variables $t_1, t_2, \ldots, t_m$ and replace each $g_j(\mathrm{x}) \leq 0$ by:

$$g_j(\mathrm{x}) + t_j^2 = 0, \quad j = 1, \ldots, m.$$

The augmented problem has $n + m$ variables (the original n plus m new ones) and m equality constraints.



The slack $s = t^2 \geq 0$ "absorbs" the gap between $g(\mathrm{x})$ and zero, converting the one-sided constraint into an exact equality.

Transformed Problem

$$\underset{\mathrm{x},\, t_1, \ldots, t_m}{\text{minimize}} \; f(\mathrm{x}) \quad \text{s.t.} \quad g_j(\mathrm{x}) + t_j^2 = 0 \;\; \forall j$$

This is now a **pure equality**-constrained problem, amenable to Lagrange multiplier methods.

Worked Example

Constraint: $g(x) = x - 5 \leq 0$.

Introduce $t \in \mathbb{R}$, set $g(x) + t^2 = 0$:

$$x - 5 + t^2 = 0 \implies x = 5 - t^2 \leq 5. \; \checkmark$$

For any real t, we get $x \leq 5$ automatically.

Slack Variables — Converting Inequalities to Equalities IV

Key Insight

Slack variables do *not* reduce the difficulty of the problem — they merely change its form. The real benefit is that standard equality-only methods (Lagrange multipliers, augmented Lagrangian) can now be applied without modification.

Penalty Methods — Enforcing Constraints by Adding a Cost I

A **penalty method** converts a constrained problem into a sequence of unconstrained problems by adding a term to the objective that is large whenever the constraints are violated. The intuition is: if violating a constraint costs a lot, the optimizer will try hard to avoid violations. As we increase the cost (the penalty weight ρ (rho)), the optimizer is forced closer and closer to feasibility.

Penalized Objective

$$\underset{x}{\text{minimize}} \; f(x) + \rho\, p(x)$$

Penalty Methods — Enforcing Constraints by Adding a Cost II

Notation and How to Read This

- ρ (rho) is the **penalty weight** (a positive scalar that controls how severely constraint violations are penalized). Small ρ allows mild violations; large ρ forces near-feasibility.
- $p(x)$ is the **penalty function** — a non-negative function that equals zero when all constraints are satisfied and is positive when constraints are violated.

Reading this formula: the optimizer minimizes both $f(x)$ (the original goal) and $\rho\, p(x)$ (the penalty for infeasibility). These two terms trade off against each other: if ρ is too small, the penalty is weak and the solution may be infeasible; if ρ is too large, the problem becomes ill-conditioned.

Penalty Methods — Enforcing Constraints by Adding a Cost III

There are several natural choices for the penalty function $p(x)$, each with different properties.

Common Penalty Functions

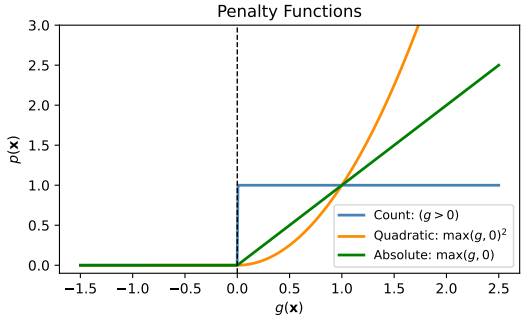
- **Count penalty:** $p(x) = \sum_j 1[g_j(x) > 0]$ counts the number of violated constraints. Each violation adds 1 to the penalty regardless of severity. This is *non-smooth* (non-differentiable) and hard to optimize with gradient methods.
- **Quadratic penalty (for inequalities):** $p(x) = \sum_j \max(g_j(x), 0)^2$. Only the *violated* constraints (where $g_j > 0$) contribute, and the contribution grows as the *square* of the violation. This is smooth everywhere.
- **Quadratic penalty (for equalities):** $p(x) = \sum_i h_i(x)^2$. Since $h_i(x)^2 \geq 0$ and equals zero iff $h_i(x) = 0$, this penalises any deviation from the equality surface.

Penalty Methods — Enforcing Constraints by Adding a Cost IV

Sequential penalty strategy. Rather than using a single large ρ, it is better to solve a sequence of sub-problems with *increasing* ρ:

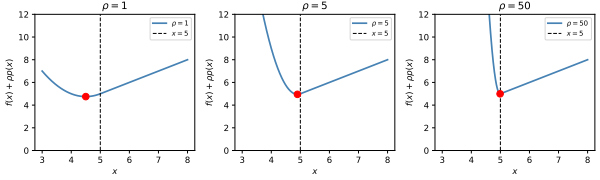
$$\rho_1 < \rho_2 < \cdots \to \infty.$$

At each stage, the solution from the previous stage initialises the next. This warm-starting approach converges more reliably than starting from scratch each time.



Different penalty functions plotted as a function of the constraint value $g(x)$. The quadratic penalty (blue curve) grows smoothly, while the count penalty (step function) is non-differentiable.

Penalty Methods — Enforcing Constraints by Adding a Cost V



Penalized objective for min x subject to $x \geq 5$, for increasing values of ρ. The minimum $x^* = 5 - 1/(2\rho)$ (x star equals 5 minus one over two rho) approaches 5 from below as $\rho \to \infty$.

Key Limitation (Example 10.5)

For the problem min x subject to $x \geq 5$, the penalized problem (with quadratic penalty for $x < 5$) has the closed-form unconstrained minimum:

$$x^* = 5 - \frac{1}{2\rho}.$$

This value is always *strictly less than 5* for any finite ρ (rho): the solution never actually reaches the constraint boundary. Exact feasibility requires $\rho \to \infty$, which causes the Hessian (second-derivative matrix) to become ill-conditioned and the solver to slow dramatically.

Practical issues with large ρ

As ρ grows, the penalized objective develops a

Python: Quadratic Penalty Method

```python
import numpy as np
from scipy.optimize import minimize

def quadratic_penalty_method(f, h, x0, k_max=20, rho=1.0, gamma=2.0):
    """
    Minimize f(x) subject to h(x) = 0 using quadratic penalty.
    f     : objective function
    h     : equality constraint function (returns array)
    x0    : initial point
    k_max: number of penalty iterations (outer loop)
    rho   : initial penalty weight (rho, pronounced 'row')
    gamma: factor by which rho is multiplied each iteration (> 1)
    """
    x = np.array(x0, dtype=float)
    for k in range(k_max):
        def penalized(xv):
            hv = h(xv)
            # Penalized objective: f(x) + (rho/2) * ||h(x)||^2
            return f(xv) + (rho / 2.0) * np.sum(hv**2)
        res = minimize(penalized, x, method='L-BFGS-B')
        x = res.x
        rho *= gamma     # increase penalty weight for next iteration
    return x

# Example: minimize x1^2 + x2^2 subject to x1 + x2 = 1
# True solution: x* = [0.5, 0.5] (closest point on the line to the origin)
f  = lambda x: x[0]**2 + x[1]**2
```

Method of Multipliers — The Best of Both Worlds I

The **method of multipliers** (also called the **augmented Lagrangian method**) fixes the key weakness of the pure penalty method by combining a quadratic penalty with an estimate of the true Lagrange multiplier. The key insight is: if we had the *exact* multiplier λ^*, the augmented Lagrangian $\mathcal{L}_\rho$ would have its unconstrained minimum *exactly* at the constrained optimum x^*, even for a *finite* value of ρ. The method iteratively refines the multiplier estimate using a simple update rule.

Augmented Lagrangian Function (Eq. 10.37)

$$\mathcal{L}_\rho(\mathrm{x}, \boldsymbol{\lambda}) = f(\mathrm{x}) + \frac{\rho}{2} \|\mathrm{h}(\mathrm{x})\|^2 + \boldsymbol{\lambda}^\top \mathrm{h}(\mathrm{x})$$

Notation and How to Read This

- ρ (rho) is the penalty weight (a positive scalar), same role as in the pure penalty method.
- $\mathrm{h}(\mathrm{x}) = (h_1(\mathrm{x}), \ldots, h_\ell(\mathrm{x}))$ is the vector of all equality constraint values.
- $\|\mathrm{h}(\mathrm{x})\|^2$ (norm squared of h) is the sum of squares of the constraint values: $h_1(\mathrm{x})^2 + \cdots + h_\ell(\mathrm{x})^2$. This is the quadratic penalty term from the pure penalty method.
- $\boldsymbol{\lambda}$ (lambda bold) is the current **estimate** of the Lagrange multiplier vector.
- $\boldsymbol{\lambda}^\top \mathrm{h}(\mathrm{x})$ (lambda transpose times h) is the dot product of the multiplier estimate and the constraint vector: $\lambda_1 h_1(\mathrm{x}) + \cdots + \lambda_\ell h_\ell(\mathrm{x})$. This is the classical Lagrangian term.

Compared to the pure penalty method, the only difference is the added term $\boldsymbol{\lambda}^\top \mathrm{h}(\mathrm{x})$, which shifts the penalty "center" so that the unconstrained minimum approaches the true constrained minimum at finite ρ.

Method of Multipliers — The Best of Both Worlds III

Algorithm 10.3 — Method of Multipliers

1. **Initialize:** set $\boldsymbol{\lambda} \leftarrow 0$ (zero multiplier estimate) and choose $\rho > 0$, $\gamma > 1$ (gamma is the growth factor for ρ).
2. **Inner minimization:** find x that minimizes the augmented Lagrangian for the current $\boldsymbol{\lambda}$ and ρ:

$$\mathrm{x} \leftarrow \arg\min_{\mathrm{x}} \; \mathcal{L}_\rho(\mathrm{x}, \boldsymbol{\lambda}).$$

   This is an *unconstrained* minimization and can use any standard solver.
3. **Multiplier update:**

$$\boldsymbol{\lambda} \leftarrow \boldsymbol{\lambda} + \rho\, \mathrm{h}(\mathrm{x}).$$

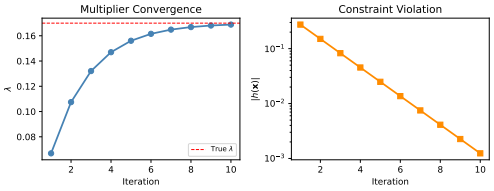
   If $h(\mathrm{x}) > 0$ (constraint violated), we increase λ, making the penalty term larger and pushing x toward feasibility.
4. **Increase ρ:** $\rho \leftarrow \gamma\rho$.
5. **Repeat** steps 2–4 until $\|\mathrm{h}(\mathrm{x})\| \leq \varepsilon$ (epsilon), where ε (epsilon) is a small convergence tolerance.

How to Read the Multiplier Update

The update rule $\boldsymbol{\lambda} \leftarrow \boldsymbol{\lambda} + \rho\, \mathrm{h}(\mathrm{x})$ is a gradient ascent step on the dual objective. Here is the intuition:

- If $h(\mathrm{x}) > 0$ (constraint violated), then λ increases. A larger λ makes the Lagrangian penalize the violation more heavily, so the next minimization step moves x closer to feasibility.
- If $h(\mathrm{x}) < 0$ (constraint over-satisfied), λ decreases, relaxing the penalty.
- At convergence, $h(\mathrm{x}) = 0$ and $\boldsymbol{\lambda}$ has converged to the true Lagrange multiplier $\boldsymbol{\lambda}^*$.

This feedback mechanism means feasibility is achieved *at finite* ρ, unlike the pure penalty method.



Left panel: multiplier estimate $\lambda^{(k)}$ (lambda superscript k) converging to the true λ^*. Right panel: constraint violation $\|\mathrm{h}(\mathrm{x}^{(k)})\|$ converging to zero.

Python: Method of Multipliers (Augmented Lagrangian)

```python
import numpy as np
from scipy.optimize import minimize

def method_of_multipliers(f, h, x0, k_max=20, rho=1.0, gamma=2.0):
    """
    Minimize f(x) subject to h(x) = 0 using the augmented Lagrangian method.
    f      : objective function
    h      : equality constraint function (returns array)
    x0     : initial point
    k_max  : maximum outer iterations
    rho    : initial penalty weight
    gamma  : penalty growth factor (> 1)
    """
    x = np.array(x0, dtype=float)
    lam = np.zeros(len(h(x)))       # initialize Lagrange multipliers to zero
    for k in range(k_max):
        def aug_lagrangian(xv):
            hv = h(xv)
            # Augmented Lagrangian: f(x) + (rho/2)||h||^2 + lambda^T h
            return f(xv) + (rho / 2.0) * np.dot(hv, hv) + np.dot(lam, hv)
        res = minimize(aug_lagrangian, x, method='L-BFGS-B')
        x   = res.x
        lam = lam + rho * h(x)      # multiplier update: lam <- lam + rho * h(x)
        rho *= gamma                # increase penalty weight
        if np.linalg.norm(h(x)) < 1e-8:
            break
    return x, lam
```

Interior Point Methods — Staying Inside by Adding a Barrier I

Interior point methods take the opposite philosophical approach from penalty methods: instead of penalizing infeasibility (staying outside), they add a **barrier** that prevents the optimizer from ever *reaching* the boundary. Every iterate is strictly feasible throughout the entire algorithm.

The core idea. For an inequality constraint $g_j(x) \leq 0$, define a function that becomes $+\infty$ (plus infinity) as $g_j(x)$ approaches zero from below (i.e., as x approaches the boundary from the interior). This infinite "wall" stops the optimizer from crossing the boundary.

Log Barrier for $g_j(x) \leq 0$

$$p(x) = -\sum_{j=1}^{m} \ln\bigl(-g_j(x)\bigr)$$

This function is defined only when all $g_j(x) < 0$ (i.e., strictly inside the feasible set).

Notation and How to Read This

- $\ln(\cdot)$ (natural logarithm) is a function that approaches $-\infty$ as its argument approaches zero from above.
- Since $g_j(x) < 0$ in the interior, $-g_j(x) > 0$, so $\ln(-g_j(x))$ is well-defined.
- As $g_j(x) \to 0^-$ (approaches zero from below), $-g_j(x) \to 0^+$, and $\ln(-g_j(x)) \to -\infty$. Therefore $-\ln(-g_j(x)) \to +\infty$: the barrier blows up, creating a wall at the boundary.
- The sum $\sum_{j=1}^{m}$ (Sigma over all m constraints) adds a wall for every inequality.

Interior Point Methods — Staying Inside by Adding a Barrier III

Barrier-Penalized Objective

$$\underset{x}{\text{minimize}} \; f(x) + \frac{1}{\rho} p(x)$$

where $\rho > 0$ (rho) controls how "thick" the barrier wall is. For small ρ the barrier is very strong (the $1/\rho$ coefficient is large), keeping the solution far from the boundary. As ρ increases, the barrier shrinks, allowing the solution to approach the boundary.

How to Read This Formula

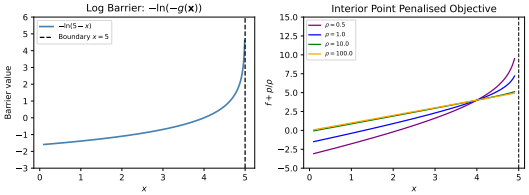
Increasing ρ (rho) makes $1/\rho$ smaller, reducing the influence of the barrier. The sequence of minimizers, as $\rho \to \infty$, traces a **central path** from the interior toward the constrained optimum on the boundary. This central-path concept is the foundation of modern interior-point methods for linear and convex programs.

Interior Point Methods — Staying Inside by Adding a Barrier IV

Other Barrier Functions

- **Inverse barrier:** $p(\mathrm{x}) = -\sum_j \frac{1}{g_j(\mathrm{x})}$. This also blows up at the boundary but has different curvature properties than the log barrier.

Both types require a **strictly feasible** starting point $\mathrm{x}^{(0)}$ with all $g_j(\mathrm{x}^{(0)}) < 0$.



Left: the log barrier blows up at the constraint boundary. Right: as ρ increases, the minimizer of the barrier-penalized problem moves toward the boundary, tracing the central path.

Algorithm 10.4 — Interior Point Method

1. Choose a **strictly feasible** starting point $x^{(0)}$ satisfying $g_j(x^{(0)}) < 0$ for all j (all constraints strictly satisfied).
2. Set $\delta \leftarrow \infty$ (delta, measuring the step size).
3. **While** $\delta > \varepsilon$ (epsilon, the convergence threshold):
   1. Minimize the barrier-penalized objective to get x': $x' \leftarrow \arg\min_x \left[f(x) + p(x)/\rho \right]$.
   2. Measure change: $\delta \leftarrow \|x' - x\|$.
   3. Update: $x \leftarrow x'$.
   4. Increase barrier: $\rho \leftarrow \gamma\rho$.
4. **Return** x as the approximate constrained minimum.

Finding an initial strictly feasible point. If no feasible point is known, solve an auxiliary problem to find one:

$$\underset{x,\, s}{\text{minimize}} \; s \quad \text{s.t.} \quad g(x) \leq s\,1$$

Interior Point Algorithm and Finding a Feasible Starting Point II

where 1 is a vector of ones and s (a scalar) is also being minimized. Start with any $x^{(0)}$ and set $s^{(0)} > \max_j g_j(x^{(0)})$ (so the pair $(x^{(0)}, s^{(0)})$ is feasible for the auxiliary problem). When this auxiliary problem finds $s^* < 0$, the corresponding x^* is strictly feasible for the original problem.

Feasibility sub-problem explained

The auxiliary problem min s subject to $g(x) \leq s1$ asks: "find the smallest s such that all constraint violations are at most s." If the minimum $s^* < 0$, then $g_j(x^*) \leq s^* < 0$ for all j, meaning x^* is *strictly* feasible for the original problem. If $s^* > 0$, the original problem is infeasible.

Pros and Cons

Advantages:

- Every iterate is strictly feasible.
- Works extremely well for convex problems (LP, QP, SOCP).
- The basis of production solvers such as IPOPT and MOSEK.

Disadvantages:

- Requires a strictly feasible starting point.

Worked Example

Problem: min $(x-3)^2$ subject to $x \geq 1$, i.e., $g(x) = 1 - x \leq 0$.

The unconstrained minimum is at $x = 3$, which satisfies $x \geq 1$. So the constraint is inactive and $x^* = 3$.

Starting from $x^{(0)} = 1.5$ (strictly feasible since $1.5 > 1$), the interior point algorithm traces a path toward $x^* = 3$ while staying above $x = 1$ throughout.

Python: Interior Point Method with Log Barrier

```python
import numpy as np
from scipy.optimize import minimize

def interior_point_method(f, g_list, x0, rho=1.0, gamma=2.0, eps=1e-6):
    """
    Minimize f(x) subject to g_j(x) <= 0 for all g in g_list.
    f       : objective function
    g_list  : list of inequality constraint functions (each g_j(x) <= 0)
    x0      : STRICTLY feasible initial point (all g_j(x0) < 0)
    rho     : initial barrier scaling parameter
    gamma   : factor by which rho grows each iteration (> 1)
    eps     : convergence tolerance (epsilon)
    """
    x = np.array(x0, dtype=float)
    delta = np.inf
    while delta > eps:
        def penalized(xv):
            # Log barrier: -sum_j log(-g_j(x)), blows up at boundary
            barriers = [-np.log(-g(xv)) for g in g_list]
            if any(np.isnan(b) or np.isinf(b) for b in barriers):
                return 1e20   # infeasible point: return a very large value
            return f(xv) + sum(barriers) / rho
        res = minimize(penalized, x, method='Nelder-Mead',
                       options={'xatol': 1e-9, 'fatol': 1e-9, 'maxiter': 5000})
        delta = np.linalg.norm(res.x - x)
        x = res.x
        rho *= gamma    # reduce barrier weight, allowing approach to boundary
```

How to Choose a Constrained Optimization Method I

With five distinct approaches now in hand, it is important to understand their trade-offs so you can select the right tool for a given problem. The choice depends on the type of constraints (equality vs. inequality), whether the objective and constraints are smooth, and whether feasibility of every iterate is required.

Method	**Constraint type**	**Feasibility**	**Key parameter**	**Note**
Substitution	Equality (affine)	Always exact	—	Reduces dimension
Projection	Convex set	Always feasible	Step size α	Needs projection oracle
Penalty	Any	Approximate	$\rho \to \infty$	Ill-conditioned for large ρ
Aug. Lagrangian	Equality	Exact at finite ρ	ρ, $\boldsymbol{\lambda}$	More robust than penalty
Interior Point	Inequality	Always feasible	ρ (barrier)	Needs strict interior start

How to Choose a Constrained Optimization Method II

Practical Selection Guide

- **Simple bounds or affine equalities:** use variable substitution or projected gradient. The problem becomes unconstrained with no approximation.
- **Black-box objective f (no gradient):** use penalty methods or augmented Lagrangian with derivative-free inner solvers.
- **Convex / LP / QP:** use interior point methods (via CVXPY or IPOPT). They exploit convexity for polynomial-time guarantees.

SciPy's Built-in Constrained Solvers

`scipy.optimize.minimize` supports constrained optimization directly via the following methods:

- `method='SLSQP'` — Sequential Quadratic Programming; handles both equality and inequality constraints; good general purpose choice.
- `method='trust-constr'` — trust-region based; more accurate gradient/Hessian use; suitable for larger problems.

Both accept a `constraints=` argument that is a list of dictionaries specifying each h or g.

Python: Using scipy.optimize.minimize with Constraints

```python
import numpy as np
from scipy.optimize import minimize

# Problem: Minimize f(x) = (x1 - 1)^2 + (x2 - 2.5)^2
# subject to: h(x) = x1 + x2 - 3 = 0   (equality constraint)
#             g(x) = x1 - x2 + 1 <= 0   (inequality constraint)
#             x1, x2 >= 0                (bound constraints)
# True solution (by hand): x* approximately [1.4, 1.7]

f = lambda x: (x[0]-1)**2 + (x[1]-2.5)**2

# scipy uses 'ineq' for constraints of the form fun(x) >= 0,
# so we negate the inequality g(x) <= 0 to get -g(x) >= 0.
constraints = [
    {'type': 'eq',   'fun': lambda x: x[0] + x[1] - 3.0},       # h(x) = 0
    {'type': 'ineq', 'fun': lambda x: -(x[0] - x[1] + 1.0)},    # -g(x) >= 0
]
bounds = [(0, None), (0, None)]     # x1 >= 0, x2 >= 0

res = minimize(f, [0.5, 2.0], method='SLSQP',
               constraints=constraints, bounds=bounds)
print(f"x* = {res.x}")
print(f"Equality residual: {res.x[0] + res.x[1] - 3:.2e}")
# x* approximately [1.4, 1.7],  equality residual = 0.00e+00
```

Listing 8: scipy.optimize.minimize with equality and inequality constraints (SLSQP)

Chapter 10 — Summary I

This chapter developed a complete toolkit for **constrained optimization** — the task of minimizing an objective $f(\mathrm{x})$ while requiring that the solution x satisfy equality constraints $h_i(\mathrm{x}) = 0$ and inequality constraints $g_j(\mathrm{x}) \leq 0$.

Fundamental Concepts

- Constraints restrict the design space to a **feasible set** $\mathcal{X} \subset \mathbb{R}^n$. They may or may not change the optimal solution, depending on whether the unconstrained optimum is already feasible.
- There are two fundamental constraint types: equality $h(\mathrm{x}) = 0$ (pinning x to a surface) and inequality $g(\mathrm{x}) \leq 0$ (pinning x to one side of a surface). All other forms (bounds, boxes, greater-than, set-membership) reduce to these two.
- **Variable transformations** (e.g., $x = a + (b - a)\sin^2 \hat{x}$) can eliminate bound constraints entirely, turning the problem into an unconstrained one.
- **LQ decomposition** removes affine equality constraints $\mathrm{Ax} = \mathrm{b}$ by splitting variables into determined and free components, reducing the effective dimension from n to $n - m$.
- **Projected gradient descent** extends unconstrained gradient descent to convex feasible sets by projecting each iterate back onto $\mathcal{X}$ after every gradient step.

The Theory: Lagrange Multipliers and KKT Conditions

- For equality constraints, the **Lagrange multiplier condition** $\nabla f(\mathrm{x}^*) = \lambda \nabla h(\mathrm{x}^*)$ (lambda times nabla h) is the first-order necessary condition for a constrained optimum.
- The **Lagrangian** $\mathcal{L}(\mathrm{x}, \lambda) = f(\mathrm{x}) - \lambda h(\mathrm{x})$ encodes both the objective and the constraint; setting its gradient to zero gives both the optimality condition and the constraint simultaneously.
- For inequality constraints, the **KKT conditions** add dual feasibility ($\mu_j \geq 0$, mu sub j non-negative) and **complementary slackness** ($\mu_j g_j(\mathrm{x}^*) = 0$, the product of multiplier and constraint value is zero).
- **Active constraints** ($g_j(\mathrm{x}^*) = 0$, $\mu_j > 0$) sit on the boundary; **inactive constraints** ($g_j(\mathrm{x}^*) < 0$, $\mu_j = 0$) have no influence on the solution.

Practical Algorithms

- **Slack variables** $s = t^2$ convert inequalities to equalities.
- **Penalty methods** add $\rho\, p(\mathrm{x})$ to the objective; increasing ρ (rho) forces feasibility but causes ill-conditioning.
- **Augmented Lagrangian** (method of multipliers) achieves exact feasibility at *finite* ρ by updating the multiplier estimate $\boldsymbol{\lambda} \leftarrow \boldsymbol{\lambda} + \rho \mathrm{h}(\mathrm{x})$ at each step.
- **Interior point** methods use a log barrier $-\sum_j \ln(-g_j(\mathrm{x}))$ (negative log of negative g) to stay strictly inside the feasible set while the barrier weight shrinks.

The Single Most Important Formula

The augmented Lagrangian multiplier update rule:

$$\boldsymbol{\lambda}^{(k+1)} = \boldsymbol{\lambda}^{(k)} + \rho\, \mathrm{h}\big(\mathrm{x}^{(k+1)}\big)$$

Here $\boldsymbol{\lambda}^{(k)}$ (lambda bold superscript k) is the multiplier estimate at outer iteration k, ρ (rho) is the penalty weight, and $\mathrm{h}(\mathrm{x}^{(k+1)})$ is the vector of constraint violations at the current solution. This simple formula drives the constraint violation to zero without requiring $\rho \to \infty$.

Exercise 10.1 — Boundary Optimum

Problem: Solve $\min_x\ x$ subject to $x \geq 0$.

Solution: The unconstrained minimum of $f(x) = x$ is $x \to -\infty$ (the objective decreases without bound). The constraint $x \geq 0$ (equivalently, $g(x) = -x \leq 0$) blocks this. The constrained minimum is $x^* = 0$, the smallest feasible value. This is an *active* constraint: $g(0) = 0$ and $\mu^* > 0$.

Exercise 10.5 — Augmented Lagrangian vs. Pure Penalty

Problem: What is the advantage of the method of multipliers over the quadratic penalty method?

Solution: The pure penalty method requires $\rho \to \infty$ to achieve a feasible solution (as shown by $x^* = 5 - 1/(2\rho)$ approaching 5 only in the limit). Large ρ causes the Hessian condition number to grow, making the sub-problems hard to solve numerically. The method of multipliers achieves exact feasibility at a *finite* ρ, because the multiplier update $\boldsymbol{\lambda} \leftarrow \boldsymbol{\lambda} + \rho\, \mathrm{h(x)}$ adjusts the center of the penalized objective so that its unconstrained minimum coincides with the constrained minimum, even for moderate ρ.

Selected Exercises with Full Solutions II

Exercise 10.6 — When to Use Interior Point

Problem: When would you use the barrier (interior point) method instead of the penalty method?

Solution: Use the interior point method when every iterate must remain strictly feasible. This is critical when: (a) the objective function f or the constraint functions g_j are undefined or singular outside the feasible set (e.g., logarithms of physical quantities that must stay positive), or (b) evaluating f at an infeasible point is costly or dangerous (e.g., in hardware-in-the-loop optimization or safety-critical systems). The barrier keeps all iterates strictly inside $\mathcal{X}$ by design.

Exercise 10.11 — Constraint That Changes the Optimum

Problem: Give a quadratic objective with two variables where adding a linear constraint changes the optimum.

Solution: Take $f(x_1, x_2) = x_1^2 + x_2^2$ with the constraint $x_1 \geq 1$. The unconstrained minimum is $\mathrm{x}^* = (0, 0)$, which lies outside the feasible set $\{x_1 \geq 1\}$. The constrained minimum is $\mathrm{x}^* = (1, 0)$ — the closest point in the feasible half-plane to the origin. The constraint is active: $g(1, 0) = 1 - 1 = 0$.

Exercise 10.12 — Substitution for Linear Equality

Problem: Minimize $x_1^3 + x_2^2 + x_3$ subject to $x_1 + 2x_2 + 3x_3 = 6$.

Solution: Use substitution to eliminate x_1. From the constraint: $x_1 = 6 - 2x_2 - 3x_3$. Substitute into the objective:

$$\min_{x_2, x_3} (6 - 2x_2 - 3x_3)^3 + x_2^2 + x_3.$$

This unconstrained problem in two variables can be solved with any standard method. The constraint is automatically satisfied for any (x_2, x_3) because x_1 is defined to satisfy it.

Exercise 10.16 — Lagrange Multiplier Degenerate Case

Problem: Why can Lagrange multipliers fail for $\min f(x)$ s.t. $c(x)^2 = 0$?

Solution: The constraint $c(x)^2 = 0$ is equivalent to $c(x) = 0$, but its gradient is $\nabla[c^2] = 2\,c(x)\,\nabla c(x)$. At any feasible point, $c(x) = 0$, so this gradient equals 0. The Lagrange condition $\nabla f = \lambda \cdot 0 = 0$ would require $\nabla f = 0$ regardless of λ, providing no information. The **constraint qualification** (the requirement that constraint gradients be non-zero at the optimum) is violated. Always use the non-squared form $c(x) = 0$ instead.

End of Chapter 10

Constraints

Algorithms for Optimization, 2nd ed.
Kochenderfer & Wheeler (2026)